

Fleet Manager

Mimarisi ve Algoritmalar

Bu düzeltilmiş sürüm, sağlanan PDF ve ilk düzeltme notunu ve ikinci turdaki yedi maddeyi temel alır. Mimari sözleşmesini tanımlar; uygulama kodunun değiştirilmiş veya çalıştırılarak doğrulanmış olduğunu ifade etmez. Dört belge aynı config, zaman ve API kararlarını kullanır.

1. Katmanlar ve sorumluluklar

React / dış istemci -> FastAPI / ROS servis adaptörü
-> command / event queue -> tek Application + Domain consumer
-> SQLite transaction + outbox
-> Nav2 dispatcher / immutable snapshot / WebSocket
Nav2 sonuçları + robot telemetrisi -> event queue

Nav2 hedefe ulaşmayı; Fleet Manager görev seçimi, yaşam döngüsü, retry, şarj ve recovery kararlarını yönetir. Simülatör SoC üretir. Robot Registry kimlik, uygunluk ve telemetriyi; Task Queue görev sırasını; Assignment Engine filtre ve seçimi; Charging Policy iç şarj operasyonlarını tutar.

Domain içinde rclpy, ROS mesajı, FastAPI veya Gazebo bağımlılığı yoktur. Adaptörler sade domain nesneleri üretir. FastAPI aynı process içindeki application katmanına erişir; kendi içinde ROS servisi üzerinden dolaşmaz.

Çalıştırma sözleşmesi

Tek state sahibi ve bellek replay tamponu için FastAPI tek worker, tek uygulama replica ile çalışır. ROS executor ve API event loop ayrı thread kullanabilir; state yalnız tek consumer tarafından değiştirilir. Çok worker ancak dış state/broker mimarisiyle mümkündür ve MVP dışıdır.

Tüm ayarlar ortak config/fleet.yaml içindedir. Robot sayısı, model ve profiller configten gelir. Bu sürümde eski Wh enerji modeli kaldırılmış, Battery Simulator ile ortak 0-1 tüketim modeli kullanılmıştır. Atama, trafik rezervasyonu veya küresel çarpışmasız rota planlama garantisi sağlamaz.

Veri modeli ve değişmez kurallar

Nesne	Alanlar
Task	id, request_id, pickup/dropoff pose, priority, state, assigned_robot_id, payload_state, stage_retry_count, requeue_count, zamanlar, entity_version
Robot	id, state, pose, battery_soc, active_task_id, telemetry_at, nav_goal_id, charge_phase, payload_state, incident_id, entity_version, deleted_at
Event / Command	event_id / command_id, request_id, robot_id, task_id, goal_id, attempt_id, manager_epoch, observed_at, payload
ChargeOperation	robot_id, station_id, phase, goal_id, deadline, retry_count, policy_revision, captured_release_level
ExperimentRun	id, scenario_id, seed, strategy, config_hash, status, sim başlangıç/bitiş, metrics, failure_reason

2. Tutarlılık

Bir robot en fazla bir aktif teslimat taşır; görev en fazla bir robota bağlıdır. Atama, robot-görev bağı, task event ve outbox aynı SQLite transaction içinde yazılır. Terminal durumlar COMPLETED, FAILED ve CANCELLED tekrar çalıştırılmaz; yeni görev veya açık recovery gerekir.

Task.state teslimat aşamasının ana kaynağıdır. Robot.state operasyonel yansımasıdır; OFFLINE, ERROR ve şarj ayrıca uygunluğu sınırlar. payload_state = NONE / ONBOARD / UNKNOWN. load_timer_done ONBOARD; unload_timer_done NONE üretir. Fiziksel sistem timer yerine sensör/operatör onayı gerektirir.

FAILED görevde yük robot üzerinde kalabilir. active_task_id temizlense bile incident ve payload bağı korunur. POST /api/robots/{id}/resolve ancak duruş, eski goal sonlanması, güncel telemetri ve yükün güvenli çözüldüğüne ilişkin operatör kaydıyla (deneyde yetkili runner doğrulamasıyla) robotu serbest bırakır. Körlemesine IDLE ataması yapmaz.

Pose ortak map frame ve ölçüm zamanı taşır. Silinen robot deleted_at ile geçmişte korunur; runtime registryden çıkması istatistiklerini veya görev referanslarını silmez. Aynı id yeniden eklenirse aynı tarihsel kimlik kabul edilir.

Görev ve robot durum makineleri

QUEUED -> ASSIGNED -> TO_PICKUP -> LOADING
 LOADING --load_timer_done--> TO_DROPOFF -> UNLOADING
 UNLOADING --unload_timer_done--> COMPLETED
 Hata / iptal: FAILED / CANCELLED
 Pickup öncesi kontrollü yeniden atama: QUEUED

Olay	Geçiş ve yan etki
task_submitted	Yeni request_id için QUEUED; aynı anahtar aynı içeriği tekrar yaratmaz.
robot_selected / goal_accepted	ASSIGNED kalıcılaştır; pickup goal kabulüyle TO_PICKUP.
pickup goal_succeeded	LOADING; /clock tabanlı bloklamayan timer.
load_timer_done	ONBOARD; TO_DROPOFF; dropoff goal gönder.
dropoff goal_succeeded	UNLOADING; boşaltma timeri.
unload_timer_done	NONE; COMPLETED; IDLE veya şarj operasyonu.
battery_low / critical	Low: charge_pending; kritik: güvenli duruş ve aktif görev FAILED. NONE -> TEMP_UNAVAILABLE; ONBOARD/UNKNOWN -> ERROR.
heartbeat_lost	OFFLINE; eski goal/ownership karantinası. Yüklü veya belirsiz görev FAILED.

Robot yansımaları ASSIGNED, NAVIGATING_TO_PICKUP, LOADING, NAVIGATING_TO_DROPOFF, UNLOADING şeklindedir. Dropoff gönderim aralığı nav_phase = SENDING / ACTIVE / RETRY_WAIT ile izlenir. Gecikmiş sonuç yalnız goal_id + attempt_id + manager_epoch eşleşirse uygulanır.

Robotun bağımsız durumları: IDLE, OFFLINE, TEMP_UNAVAILABLE, ERROR, CHARGING. TEMP_UNAVAILABLE yüksüz geçici hata için otomatik toparlanır; ERROR yük/incident belirsizliği için kullanılır. Şarj fazları TO_STATION / DOCKING / CHARGING / RELEASING. TO_STATION fiziksel şarj değildir. OFFLINE -> IDLE için taze telemetri, uzlaştırılmış goal ve çözülmüş yük/hata gerekir; tek heartbeat yeterli değildir.

Yüzde bazlı enerji yeterliliği

3. Simülatör ile aynı katsayılar

$S_{delivery} = a_{move} * t_{drive_delivery} + k_{meter} * d_{delivery} + a_{idle} * (t_{load} + t_{unload} + t_{wait_delivery})$
 $S_{return} = a_{move} * t_{drive_return} + k_{meter} * d_{return} + a_{idle} * t_{wait_charger}$
 $S_{required} = (1 + uncertainty) * (S_{delivery} + S_{return})$
 $S_{after} = SoC - S_{required}$
 Uygunluk: $S_{after} > safety_reserve$

a_{move} = moving_drain_per_sec; a_{idle} = idle_drain_per_sec; k_{meter} = consumption_per_meter.

Sonuçların tümü SoC kesridir. Wh, kapasite veya yardımcı güç terimi eklenmez. Mesafe terimi simülatördeki gibi hareket zaman tüketimine ilavedir. Profil, battery.models üzerinden ortak okunur.

Rota robot -> pickup -> dropoff -> uygun şarj istasyonudur. Erişilebilir yol uzunluğu metre, süre saniyedir; bekleme süreleri idle tüketimine eklenir. Geçerli yol yoksa veya telemetri bayatsa aday reddedilir. Öklid mesafesi yalnız hızlı ön elemedir; engelli haritada kesin yeterlilik kararı değildir.

Örnek	Sonuç
SoC=0.30; teslimat=0.12; dönüş=0.06	uncertainty=0.15 -> ihtiyaç 0.207.
Kalan=0.093; safety_reserve=0.10	0.093 > 0.10 yanlış: görev atanmaz.
SoC=0.22; low_level=0.25	Enerji tahmininden bağımsız işletim filtresinde elenir.

Robot IDLE, online, taze, yüksüz ve hatasız olmalı; SoC > low_level ayrıca sağlanmalıdır. İstasyon erişimi ve kapasitesi hesaba katılır. MVP robot başına istasyon veya basit rezervasyon kullanır; sıra beklemesi varsa tahmine eklenir. Bilinmeyen bekleme süresi iyimser sıfır kabul edilmez.

uncertainty ≥ 0, D_ref/S_ref/T_ref/L_ref > 0; safety_reserve örneği 0.10. Katsayılar ve referanslar fleet.yaml içinde; deneye kalibre edilir. Simülatörün SoC güncellemesi ile bu ileriye dönük yeterlilik tahmini ayrı sorumluluklardır.

Atama stratejileri ve maliyet

4. Ortak filtre, farklı seçim

Strateji	Kural
Random	Uygun adaylar arasından uniform seçim; RNG seed kaydedilir.
Nearest	En kısa erişilebilir robot -> pickup yolunu seçer.
CostBased	Normalize maliyetin en küçüğünü seçer.

$$J(r,t) = wd * d_{pickup} / D_{ref}$$

$$+ we * S_{required} / S_{ref}$$

$$+ wt * ETA_{delivery} / T_{ref}$$

$$+ wb * safety_{reserve} / S_{after}$$

$$+ wl * workload / L_{ref}$$

$$wd + we + wt + wb + wl = 1; \text{ her ağırlık } \geq 0$$
 Varsayılan: $wd=.35, we=.30, wt=.20, wb=.15, wl=0$

Dört temel terim mesafe, tüketim, ETA ve batarya marjıdır. İş yükü opsiyonel beşinci terimdir; varsayılan 0. workload aynı ölçüm penceresinde robotun birikmiş aktif görev süresidir; L_{ref} saniyedir. Yalnız IDLE adaylar olduğundan aktif görev adedini kullanmak ayırt edici olmaz. Referans ölçekler aday kümesine göre değişmez.

```

assign(task):
    candidates = filter_idle_fresh_empty_and_energy_sufficient()
    selected = strategy.select(task, candidates)
    if selected is None: return KEEP_QUEUED
    transaction:
        recheck_availability_and_entity_versions()
        bind_task_robot(); save_event_and_outbox()
    dispatch_pickup_after_commit()
  
```

Görev sırası priority azalan, created_at artandır; aging opsiyoneldir. Eşit puanlarda robot_id ile deterministik bağ kırılır. Requeue için ayrı limit ve cooldown uygulanır. select_robot(task, candidates, context) robot_id veya None döndürür; domain ROS nesnesi almaz. Bu çevrimiçi greedy yaklaşım küresel optimum garantisi vermez.

Batarya, retry ve iptal

5. Şarj politikası

Örnek eşikler emergency_level=0.08, low_level=0.25, charge_release_level=0.80, safety_reserve=0.10. $0 \leq \text{emergency} < \text{low} < \text{idle_charge_level} \leq \text{release} \leq 1$; $\text{safety_reserve} \geq \text{emergency_level}$. $\text{SoC} \leq \text{low}$ yeni görevi engeller; boş robot şarja gider. Aktif teslimatta charge_pending işaretlenir. $\text{SoC} \leq \text{emergency}$ veya kalan rota yetersizliği güvenli duruşu tetikler; NONE yükte TEMP_UNAVAILABLE, ONBOARD/UNKNOWN yükte ERROR uygulanır.

Şarj oturumu release eşiğine kadar kilitlidir; %26 olması %80 hedefini kaldırmaz. Zone + stationary + stable dock_contact şartları simülatörde doğrulanır. Bayat BatteryState, contact kaybı veya dock/charge timeout başarı sayılmaz. Hedefe erişimden sonra undock doğrulanınca IDLE yapılır. Oturum politika sürümünü başlangıçta yakalar.

6. Retry ve yük güvenliği

Durum	Karar
Pickup öncesi Nav2 hatası	Aynı hedefte 2 ek retry: toplam 3 girişim/aşama. Limitte robot TEMP_UNAVAILABLE; payload NONE ve eski goal durmuşsa bütçeli requeue. Cooldown + taze telemetri + Nav2 hazır kontrolü sonrası otomatik IDLE.
LOADING belirsizliği	UNKNOWN; FAILED + ERROR; otomatik yeniden atama yok.
Pickup sonrası hata	Aynı robot/dropoff için retry; limitte FAILED, ONBOARD ve incident korunur.
UNLOADING belirsizliği	Yük boşaltıldığı doğrulanmadıysa UNKNOWN/ONBOARD; operatör çözümü.
İletişim/timeout	Cancel/terminal ve duruş doğrulanmadan yeni goal veya robot ataması yok.

max_navigation_retries=2 ilk girişimden sonraki ek denemelerdir. stage_retry_count yeni aşamada sıfırlanır; requeue_count ayrı bütçedir. Backoff /clock timerıyla uygulanır, callback içinde sleep yoktur. Aktif retry dizisi yakaladığı limiti korur.

POST /api/tasks/{id}/cancel -> 202 komut kabulüdür. Pickup sonrası ve LOADING/UNKNOWN belirsizliğinde 409. QUEUED doğrudan, ASSIGNED/TO_PICKUP ise goal duruşu doğrulanınca CANCELLED olabilir. Cancel isteği ile yükleme geçişi aynı consumer içinde sıralanır; yarışta durum tekrar doğrulanır.

Recovery, kalıcılık ve threading

7. Yeniden açılışta atama kapısı kapalı

Kayıtlı aşama	Recovery
QUEUED	Korunur; uygun robot bekler.
ASSIGNED / TO_PICKUP	Payload NONE ve eski goal durmuşsa QUEUED; aksi halde karantina.
LOADING	FAILED, UNKNOWN, robot ERROR.
TO_DROPOFF	FAILED; ONBOARD bağı korunur; açık operatör recovery gerekir.
UNLOADING	FAILED; yük bırakma teyidi; kör pickup tekrarı yok.
Terminal	Kayıt ve incident korunur.

Başlangıçta manager_epoch ve stream_id yenilenir; robotlar OFFLINE kabul edilir. Telemetri, goal ve yük uzlaştırılır; recovery transaction tamamlandıktan sonra yalnız uzlaştırılmış robotlar için atama açılır.

8. SQLite ve dış etkiler

tasks, robots, task_events, commands, outbox, incidents, experiment_runs ve robot_metrics tabloları tek writer ile güncellenir. State + journal + dış etki niyeti aynı transactiondadır. ROS action gönderimi DB ile atomik değildir; outbox tekrarlarında goal/attempt uzlaştırması kullanılır. Exactly-once varsayılmaz.

API request_id veya Idempotency-Key kalıcı benzersiz kayda bağlanır. Aynı anahtar ve aynı normalize gövde aynı task/command sonucunu döndürür; farklı gövde 409 üretir. command_id sunucu izleme kimliğidir, istemci tekrar önleme anahtarının yerini almaz.

9. Kuyruk sınırları

API ve ROS callbackleri state yazmaz; bounded thread-safe kuyruklara gönderir. Consumer adil sırayla işler, transaction sonrası immutable snapshot ve yayın üretir. Kritik goal/cancel/policy olayları sessizce düşmez; backpressure veya journal uygulanır. Mesafe entegrasyonu ve batarya eşik olayları telemetri coalesce işleminden önce çıkarılır.

API tek worker/replica; UI ve ROS bağlantı canlılığı monotonic duvar saatiyle, görev süreleri /clock ile izlenir. Simülasyon pause durumunda bağlantı heartbeat devam eder. Saat reseti eski timer ve odom tabanını geçersiz kılar.

İstatistikler ve deney runner

10. Tek ölçüm sözleşmesi

Metrik	Tanım
distance_travelled	Ham ardışık geçerli odomdan coalesce öncesi entegre metre; frame/reset sıçraması dışlanır.
completed_tasks / failed_tasks	Görev tablosundan türetilir; ayrı bağımsız sayaç tutulmaz.
active_time	ASSIGNED, TO_PICKUP, LOADING, TO_DROPOFF, UNLOADING aralıklarının /clock süresi; şarj hariç.
charging_count	Doğrulanmış fiziksel şarja başlayan oturum sayısı; status titreşiminde artmaz.
consumption_soc	Şarj kazancı düşülmeden, pozitif deşarj artımlarının toplamı; Wh değildir.

Kalıcı toplam ve deney/run bazlı metrikler ayrıdır. Robot soft delete geçmiş toplamlarını korur. Görev sonuçları assigned_robot_id ile ilişkilendirilir; requeue girişimleri event geçmişinde ayrıca izlenir. Zaman reseti metriklerde negatif süre üretmez.

11. CLI / batch deney akışı

Runner girdisi: scenario dosyası + seed + strategy. Senaryo; harita/config hash, görev dizisi ve varış zamanları, başlangıç pose/SoC, istasyon kapasitesi, bitiş koşulu ve maksimum simülasyon süresini içerir. Senaryo dosyası deney girdisidir; ikinci runtime config kaynağı değildir.

Her koşu öncesi deney kilidi alınır; canlı task/cancel/resolve ve config mutasyonları 409 experiment_running döner. Runner görev komutları ve resolve iç yetkisiyle ilerler; yük çözümü simülasyon adaptörü tarafından doğrulanır. Simülasyon, pose, SoC, goal, timer, RNG ve run sayaçları resetlenir; TF/Nav2/batarya hazırlık bariyeri geçilir. Başarısız reset koşuyu FAILED yapar.

Random, Nearest ve CostBased aynı başlangıç koşulları, görev dizisi ve uygunluk filtresiyle çalışır; Random birden fazla seed ile tekrarlanır. Süreler /clock üzerinden ölçülür. Durmuş simülasyonu yakalayan operasyon watchdog ayrı duvar saatidir.

SQLite koşu sonucu ve metrikleri saklar; CSV dışa aktarılır. completion rate, wait/delivery süresi, distance, consumption_soc, retry ve charge sayısı raporlanır. Web MVP yalnız GET /api/experiments ve GET /api/experiments/{id} ile sonuç gösterir; webden başlatma stretch kapsamıdır.

Arayüzler ve kabul senaryoları

12. Paket ve taşıma sözleşmesi

fleet_interfaces: ROS msg/srv; fleet_manager: domain, application, assignment, ros_adapters, persistence, api; battery_simulator: yüzde modeli; fleet_sim: world/URDF/launch; frontend: React. Paylaşılan config/ tek dizindir.

Standart arayüzler BatteryState, PoseStamped ve NavigateToPose. RobotStatus robot_id/state/charge_phase/SoC/pose/payload/active_task taşır; TaskStatus aşama, retry, payload ve version taşır. SubmitTask ve CancelTask request_id alır. Strateji değişimi de ortak taslak/Apply servisine gider; configi atlayan ayrı setter yoktur.

HTTP: /api/tasks, /api/tasks/{id}/cancel, /api/robots/{id}/resolve, /api/fleet/status, /api/map, /api/locations, /api/config ve robot CRUD, /api/config/apply, deney sorguları. WebSocket /ws/fleet: stream_id, replay sequence, tam entity nesnesi ve ayrı geçici telemetri. Ayrıntılı sözleşme Web Panel v1.2 FIX belgesindedir.

13. Kabul kontrol listesi

Enerji yetersiz robot QUEUED işi alamaz. Pickup öncesi üç başarısız girişim kontrollü requeue; pickup sonrası limit FAILED + yük bağı üretir. Şarj temassız başlamaz; %80 oturumu %79'da serbest kalmaz. LOADING restartı robotu otomatik IDLE yapmaz.

Eski goal sonucu ve yinelenen request_id çift görev üretmez. Telemetri seyreltilse bile mesafe değişmez. Silinen robotun geçmişi kaybolmaz. Aynı senaryo ve seed başlangıç koşulları tekrarlanabilir olmalı; deney sırasında kullanıcı mutasyonları reddedilmelidir.

Snapshot/WS yarışı replay ile kapanmalı; akış kimliği değişince resync istenmelidir. Config apply hatasında running_revision doğru kalmalıdır. Kontroller bu mimarinin uygulama kabul ölçütleridir; mevcut kodun test sonucu olarak sunulmaz.

Kaynak: sağlanan Fleet Manager Mimarisi ve Algoritmalar v2.0 (13 sayfa); düzeltme notu A1-C17. Referanslar:
<https://fastapi.tiangolo.com/deployment/concepts/> ;
https://github.com/ros2/common_interfaces/blob/rolling/sensor_msgs/msg/BatteryState.msg ;
https://github.com/ros-navigation/navigation2/tree/main/nav2_msgs/action

Fırsat şarjı ve otomatik toparlanma

Fırsat şarjı: IDLE ve $SoC < idle_charge_level$ (0.50) ise veya ulaşılabilir bir bekleyen görev yalnız enerji yeterliliği nedeniyle eleniyorsa şarj planlanır. $low < idle_charge_level \leq release$ doğrulanır. Örnekte $SoC=0.30$ robot, 0.307 ihtiyacı karşılayamadığında $low=0.25$ eşliğine düşmeyi beklemez. İstasyon erişimi/rezervasyonu doğrulanır; kapasite yoksa `charge_pending` görünür kalır ve yeniden değerlendirilir. FULL robotun enerji filtresinden geçemediği iş için sonsuz şarj döngüsü kurulmaz; görev `energy_infeasible` olarak görünür kalır.

Yüksüz geçici navigasyon hatası

Pickup öncesi retry limiti: `payload=NONE` ve eski goal terminal/duruş teyitliyse görev requeue bütçesiyle QUEUED olur, robot `TEMP_UNAVAILABLE` olur. `navigation.recovery_cooldown` bitince taze pose/ SoC , Nav2 action/lifecycle hazır ve eski goal kapanmış koşulları tekrar denetlenir. Başarılıysa IDLE ve yeniden atama/şarj değerlendirmesi; değilse geçici kullanılamaz kalır. Operatör resolve gerekmez.

Goal duruşu teyitsizse ownership karantinası korunur; cooldown tek başına serbest bırakmaz. Düşük batarya nedeniyle toparlanma, yeni iş yerine şarj planlayabilir. ONBOARD/UNKNOWN olaylarında ERROR + incident korunur; normal çalışma modunda açık operatör çözümü gerekir.

Deneyde yetkili resolve

Runner iç yetkileri: görev gönderme, reset ve resolve. ONBOARD/UNKNOWN hatası önce FAILED teslimat ve incident olarak kalıcı kaydedilir. Runner simülasyon adaptörüne güvenli duruş + yükü kaldırma/reset işlemi uygulatır; goal terminal, `payload NONE` ve taze telemetri/Nav2 hazır teyidi aldıktan sonra resolve ile serbest bırakır. FAILED görev başarıya çevrilmez, `failed_tasks` metriğinde kalır; çözüm ayrıca audit olayıdır.

Bu yetki yalnız izole simülasyon deney modunda ve `run_id` ile kullanılabilir; canlı robotta otomatik yük boşaltma anlamına gelmez. Duruş/yük temizliği doğrulanamazsa robot atama dışı kalır ve koşu süre limiti/watchdog ile FAILED tamamlanır; süresiz operatör beklemez.

Kabul senaryoları

$SoC=0.30$ ve $ihtiyaç=0.307 \rightarrow$ fırsat şarjı. Yüksüz üç Nav2 hatası $\rightarrow$ `TEMP_UNAVAILABLE` $\rightarrow$ cooldown ve hazır kontrolü $\rightarrow$ IDLE. Yüklü hata $\rightarrow$ FAILED + kontrollü runner resolve, failure metriği korunur. Live strateji Apply görev sırasında kabul; Restart-Apply 409; kuyruktaki işler korunur ve eski epoch sonucu elenir.

Parametre ve yayın sözleşmesi

Tüm eksik parametrelerin somut yolları Config v1.3 FIX ek şemasındadır:

assignment.uncertainty/references/workload_window;
tasks.retry_backoff/max_requeues/requeue_cooldown; navigation.recovery_cooldown;
battery.policy.dock_timeout/charge_timeout; battery.docking; fleet.health/web/map/stations/locations.

Fırsat şarjı idle_charge_level=0.50 ve safety_reserve $\geq$ emergency_level ile doğrulanır. Eşikler ve timeoutlar yeni oturum/karar sınırında yakalanır. Geometri, kaynak topicler, heartbeat/telemetry yayın eşikleri ve buffer kapasitesi Restart sınıfıdır. /<id>/pose_robot_pose_adapter; /<id>/dock_contact simulation_adapter üretir.

Replay yükünü sınırlama

robot_update yalnız ayrı RobotState tam nesnesini taşır: id, state, active_task_id, goal, payload_state, incident, charge_phase, entity_version. Yalnız bu alanlar değiştiğinde entity_version ve replay sequence artar. Pose/SoC/battery_status robot_telemetry kanalındadır. distance_travelled ve active_time gibi sürekli istatistikler REST Robot Details sorgusunda sunulur; değişimleri replay olayı üretmez.

GET /api/robots/{id} ayrı state + son telemetry + stats birleşimini döndürür; entity_version birleşik nesnenin tüm alanlarına ait sürüm değildir. telemetry_version yalnız telemetriyi sıralar. stats_at ve sim_time istatistik kesitini belirtir; UI detay açıkken düşük hızla REST yenileyebilir.

Auth: POST /api/session tokenı doğrular ve HttpOnly cookie kurar; REST ve WS aynı oturumu kullanır. Browser tokenı URL veya Bearer başlığında yeniden taşımaz. Ayrıntı Web v1.2 FIX oturum sözleşmesindedir.